ARC-AGI Milestone C

Scott Schmidl
sschmidl3@gatech.edu

***Abstract*—**In this milestone, I extend my rule-based agent from Milestone B to tackle the Milestone C problems. The agent still treats each problem as a search over a bank of candidate transformations and builders, but I have expanded the rule bank to better match the C examples. For each problem, the agent proposes hypotheses from the rule bank, filters them by requiring exact matches on all training examples, and applies the remaining hypotheses to the test grid. On the Milestone C problems, this approach achieves a strong performance on the training examples at 16/16 and 14/16 on the hidden cases. In this report, I will describe how the current agent is designed, analyze where it succeeds and fails, discuss runtime and scalability, and outline the changes I plan to make for the upcoming D problems and the final project.

1 HOW DOES YOUR AGENT CURRENTLY FUNCTION?

My agent, for Milestone C, keeps the same overall structure as in B, that is, it's rule-based, treats each problem as a form of hypothesis testing over a bank of transformations. For each problem, the agent loads all of the training pairs and the test input grid. It then iterates over the collection of candidate rules and looks for any rule that can reproduce the outputs for every training input. Any rule that fails on one or more examples is rejected. The remaining rules are then applied to the test grid to generate predictions.

Internally, the rule bank for this milestone is focused on transformations that appear often. These rules include building quadrants and tilings, transposing grids, moving and extending patterns, creating and expanding rectangles, producing staircases, and filling regions relative to markers. Many of these rules rely on geometric and object primitives implemented with shared utilities. For example, the utilities detect connected components, compute bounding boxes, identify 2×2 blocks, and find holes and borders, which the rules then use as high-level building blocks.

The agent's main loop organizes rules in passes. First, it evaluates the static rules from the bank, such as quadrant construction `quad`, a transpose rule, folding

`fold5`, reflecting `refl5`, and geometric rules like `stair`, `fill32`, and `row_fill`. Next, it considers rules that depend on the structure, such as detecting inner regions `inner4`, extending diagonals from `2 × 2 diag12`, or building a histogram layout based on color frequencies `freq_col`.

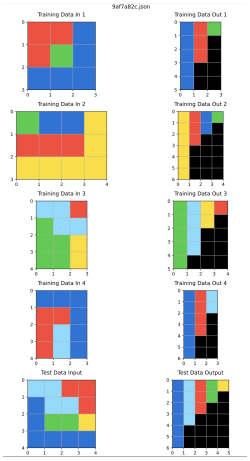


Figure 1—Colors Rotate, Sorted Desc Left to Right

For each candidate, the agent applies it to every training input and checks if the result matches the corresponding output. If it succeeds on all pairs, it is considered valid. During prediction, each valid rule is applied to the test grid, and unique outputs are collected. Only the top three predictions are kept.

Conceptually, the agent is still a search system, but the search space is finite and interpretable. This makes it easier to understand how problems are solved from input to transformation to output. Milestone C's main change is simply that the rule bank has grown and is more specialized, while preserving the same overall loop from Milestone B.

2 HOW WELL DOES YOUR AGENT CURRENTLY PERFORM?

My agent performs quite well. Using only the Milestone C rule bank and existing builders, the agent solves 16 out of 16 training pairs. The agent also does well on the hidden cases, passing 14 out of 16 of them. The two failures occur on problems `7b6016b9` and `b2862040`.

Overall, I'm happy with this performance. The agent appears to be generalizing well. It is able to discover consistent rules that explain training pairs and apply them to new cases. At the same time, the failures on the hidden cases suggest

that some of my rules may still assume too much. In the future, I would like to focus on solving these cases.

3 WHAT PROBLEMS DOES YOUR AGENT PERFORM WELL ON?

My agent performs best on problems where the transformation can be described as a structural rule that is stable across all training pairs. This often means cases where the grid can be broken into regions such as quadrants, stripes, or panels, and the output is obtained by copying, reflecting, or combining those regions. For example, the `quad` rule captures cases where the smaller pattern is replicated across quadrants, and the transpose rule handles cases where the rows and columns are swapped.

Another set of strengths comes from examples involving geometric growth or movement. The `step34` and `stair` rules catch patterns where a shape or color moves in a stepwise fashion, while `fill32` and `row_fill` expand or fill regions relative to markers. On these problems, once the agent knows the pattern it generalizes well to the test case.

The agent also does well when color plays a big role. Rules such as `inner4` and `hole8` catch frame and marker color patterns where a region's interior needs to be recolored or where holes and borders must be filled in a consistent way. In a similar way, the `freq_col` rule is used when colors are reorganized by their frequencies, producing an output that looks like a histogram. In general, when a solution can be expressed as a transformation that involves symmetry, growth, or color roles the current bank performs well.

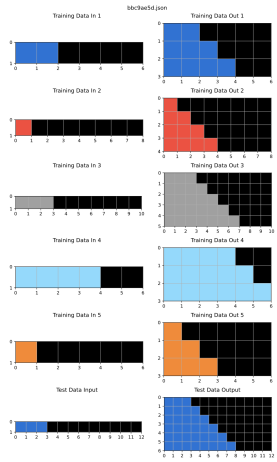


Figure 2—Output Rows Half of Input Columns. Sorted Asc Top to Bottom

4 WHAT PROBLEMS (IF ANY) DOES IT STRUGGLE ON?

Even though the agent solves the training examples and most of the hidden cases, there are still a couple failures, specifically, `7b6016b9` and `b2862040`. In these cases, my rule bank captures a pattern that fits the training pairs, but doesn't fully generalize to the broader concept. In `7b6016b9`, the pattern is to recolor the outside region green and the inside orange. My agent learns a similar behavior on training pairs, but makes too many assumptions. When the hidden case changes with the same rule, my agent applies the same recoloring but doesn't generalize to the new case. Similarly, in `b2862040`, the pattern is to change fully-connected dark blue squares to light blue ones. Once again, it finds a rule that works on the training grid, but overfits. As a result, when the hidden case changes, the rule either misses it or partially transforms the wrong regions.

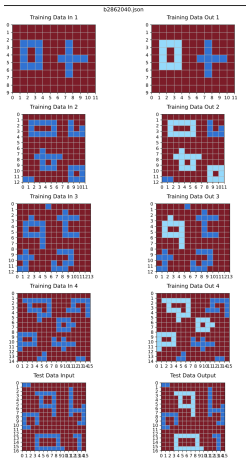


Figure 3—Change Color of Connected Squares from Dark to Light Blue

In general, the agent struggles more on problems where the correct solution is not a single transformation, but rather a composition or an abstract relationship between objects. For example, if solving a task requires identifying an object, then transforming it, and finally embedding the result into a new shape, my agent has difficulty because it only tries individual rules. A rule might explain one aspect of the training pairs while missing another constraint that's in the hidden case.

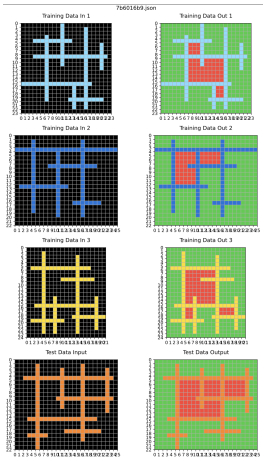


Figure 4—Fill Outside with Green and Inside with Orange

Another source of difficulty is problems that depend on relational reasoning between multiple objects and color. Many of my new rules assume specific layouts,

i.e. marker color or quadrant decomposition. When the true pattern is more flexible, i.e. a variable size block, or composition, the existing rules don't match perfectly. In those cases, the agent either falls back to a weaker rule or fails to find a rule at all. The two previously stated failures are good examples of this kind of miss where the agent understands the idea, but doesn't fully generalize.

5 HOW EFFICIENT IS YOUR AGENT?

The agent remains reasonably efficient, even though the rule bank grew. The overall structure is unchanged, i.e. for each problem the agent iterates over a finite number of candidate rules and applies them to every training input, removing rules that fail to reproduce the output. This keeps the search space small, since no rule is revisited once it has failed the training pairs.

From a complexity standpoint, the runtime is about linear, O(n), in the number of candidate rules times the total grid area across all training pairs. There is generally only one pass over the grid or connected components. This means that adding more rules increases runtime, but in a consistent way. Some rules, such as those that find connected components or detect holes and borders, use labeling and binary dilation, but these are still linear.

The runtime has remained low enough that I can iterate quickly. On Gradescope, the runtime was 0:00:00.103762 seconds, while locally it was 0.027084250 seconds. These numbers correspond to only milliseconds per problem on average. I didn't observe any individual case that caused a significant slowdown. The main risk is not necessarily the cost of a single rule, but the growth of the rule bank.

6 HOW DO YOU PLAN TO IMPROVE YOUR AGENT?

The main improvement I plan to make before the final project is to allow for more compositions of rules. The current method, while working right now, might limit the agent's ability to solve examples that decompose into multiple steps. My goal is to allow the agent to build and test compositions of rules, while keeping the search space manageable. For example, a problem might require first transposing and then applying a staircase to fill. Another might involve cropping and then applying a histogram-type layout. If I can express these as combined rules and still filter them, I should be able to cover a larger range of patterns.

A second improvement I can make is to refine some current rules, such that

they're more general. This is especially apparent with the two cases I failed on Milestone C and three I failed on Milestone B. I would like to revisit these rules to see if I can get them to notice a relative structure instead of a fixed one.

Finally, I could invest more into debugging around certain failures. I currently log which rules pass the training pairs, but not near misses that are close to working but fail on one example. Before the final project, I plan to add more detailed logging, so that I can identify whether a rule is missing a small condition or entire patterns. This should help me prioritize what rules will give me the biggest improvements.

7 HOW DO YOU PLAN TO GENERALIZE YOUR AGENT?

For Milestone D, I expect the problems to have more generalization. Many of the C problems I was able to capture with good transformations such as folding, quadrants, and staircase growth. In contrast, I expect the Milestone D problems to be more about reasoning through object roles, relationships, and composition. I will need to address these changes to be successful with the D problem set.

My plan to generalize has two parts. First, I want to broaden the representational power of the agent by using short sequences of rules as my candidates. This will let the agent incorporate more compositions, i.e. detect a shape, transform it, then embed it somewhere else. Since each single rule is still interpretable, the resulting compositions should also be interpretable. One additional thing I will have to think about is keeping the number of compositions manageable.

Second, I want to improve the way the agent describes and reasons about grids, beyond raw colors. The utilities I have are good for now, but can be expanded to capture more abstract patterns such as relationships between multiple objects in the grid. If I can accomplish both of these tasks, I believe my agent will scale to the more challenging Milestone D problems.

8 WHAT FEEDBACK WOULD YOU HOPE TO GET?

The main feedback I would like from classmates is on how to introduce rule compositions without letting the search space explode. My current method might struggle on problems that require combining two or three rules. One option is to design a way for the training pairs to suggest which combinations are most plausible and reject others.